

Vendor Performance Data Analytics Report

1. Objective

Analyze vendor performance to identify inefficiencies and optimize operations.
Provide actionable recommendations via dashboards and reports.

2. Data Sources

SQL Database: Vendor transactions and performance metrics.

Python: For data cleaning and feature engineering.

Power BI: For visualizations and reporting.

3. Methodology

Data Cleaning: Removed duplicates, handled missing values, and standardized formats using Python.

Exploratory Data Analysis (EDA): Identified trends and outliers, calculated KPIs (Key Performance Indicators) such as delivery time and defect rates.

SQL Queries: Created summary tables for aggregated insights, like vendor-wise performance stats.

Visualization: Designed Power BI dashboards with interactive elements (e.g., slicers).

4. Key Insights

Top Vendors: Ranked based on delivery time and quality scores.

Underperforming Vendors: Identified vendors with high defect rates or delays.

Trends: Seasonal variations in vendor performance were observed.

5. Recommendations

Replace or negotiate with underperforming vendors.

Incentivize top vendors to maintain quality.

Automate data monitoring for real-time insights.

6. Dashboard Highlights

KPI Overview: Delivery time, defect rates, and cost efficiency.

Vendor Comparison: Interactive chart comparing vendors across metrics.

Actionable Alerts: Highlighted vendors needing immediate attention.

7. Tools Used

SQL: For querying and aggregating data.

Python: For cleaning and transforming data.

Power BI: For creating dynamic dashboards.

8. Conclusion

This project demonstrates an end-to-end data analytics workflow, integrating multiple tools to solve real business problems. The insights and recommendations can significantly optimize vendor management.

data cleaning and feature engineering used in the project:

```
1import pandas as pd
2
3# Load dataset
4data = pd.read_csv("vendor_data.csv")
5
6# Step 1: Handle Missing Values
7data.fillna(method='ffill', inplace=True) # Forward fill for
missing values
8
9# Step 2: Create New Features
10data['Delivery_Duration'] =
    (pd.to_datetime(data['Delivery_Date']) -
    pd.to_datetime(data['Order_Date'])).dt.days
11data['Defect_Rate'] = (data['Defective_Items'] /
    data['Total_Items']) * 100
12
13# Step 3: Filter Relevant Data
14filtered_data = data[data['Delivery_Duration'] > 0] #
Remove invalid durations
15
16# Step 4: Save Cleaned Data
17filtered_data.to_csv("cleaned_vendor_data.csv", index=False)
18
```

```
19print("Data cleaning and feature engineering completed!")
```

Here's a Python script to visualize vendor performance using Matplotlib and Seaborn:

```
1import pandas as pd
2import matplotlib.pyplot as plt
3import seaborn as sns
4
5# Load cleaned dataset
6data = pd.read_csv("cleaned_vendor_data.csv")
7
8# Step 1: Bar Plot for Average Delivery Duration per Vendor
9avg_delivery =
data.groupby('Vendor')['Delivery_Duration'].mean().sort_values
()
10plt.figure(figsize=(10, 6))
11sns.barplot(x=avg_delivery.values, y=avg_delivery.index,
palette="Blues_d")
12plt.title("Average Delivery Duration by Vendor")
13plt.xlabel("Average Delivery Duration (Days)")
14plt.ylabel("Vendor")
15plt.show()
16
17# Step 2: Scatter Plot for Defect Rate vs. Delivery Duration
18plt.figure(figsize=(8, 6))
19sns.scatterplot(data=data, x='Delivery_Duration',
y='Defect_Rate', hue='Vendor', palette="Set2", s=100)
20plt.title("Defect Rate vs Delivery Duration")
```

```
21plt.xlabel("Delivery Duration (Days)")
22plt.ylabel("Defect Rate (%)")
23plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
24plt.show()
```

This script creates:

- A bar plot comparing average delivery durations for each vendor.

- A scatter plot showing the relationship between defect rates and delivery times.

Here's how the plots help analyze vendor performance:

Bar Plot:

Shows each vendor's **average delivery duration**.

Helps identify vendors with consistently slow or fast deliveries.

Example: Vendor A averages 3 days, but Vendor B takes 10 days—more delays mean inefficiency.

Scatter Plot:

Plots **defect rate vs. delivery duration** for each vendor.

Reveals patterns, like if longer delivery times correlate with higher defect rates.

Example: A vendor with long delivery times and high defects needs immediate attention.

Here's a detailed breakdown of the scatter plot:

Purpose:

It visualizes the relationship between **delivery duration** (x-axis) and **defect rate** (y-axis) for each vendor.

Each point represents a vendor's performance for a specific transaction.

Insights:

Vendors with **short delivery durations** and **low defect rates** are ideal.

If points cluster at high values (e.g., long durations + high defects), those vendors require attention.

Color Coding:

Different colors (via `hue= 'Vendor '`) help distinguish vendors.

Example: If Vendor A clusters in the top-right, they're underperforming.

Summary

This project simulates a professional data analyst workflow, integrating SQL for data extraction and transformation, Python for exploratory and statistical analysis, and Power BI for interactive reporting. The goal is to identify high-performing and underperforming vendors based on metrics like profit margins, delivery efficiency, and cost management.

Highlights

Project Introduction & Tools Overview

- Explains the business problem: optimizing vendor selection and performance.
- Introduces the tech stack: SQL, Python, Power BI.

SQL Data Preparation

- Cleans and joins raw tables: purchases, sales, invoices, product prices.
- Creates summary tables for analysis.

Python Exploratory Data Analysis (EDA)

- Detects outliers, missing values, and negative profit margins.
- Visualizes trends in freight costs, stock turnover, and vendor reliability.

Vendor Performance Metrics

- Calculates KPIs: gross profit, profit margin, delivery delays, cost ratios.
- Ranks vendors and identifies improvement areas.

Power BI Dashboarding

- Builds interactive dashboards with vendor scorecards.
- Enables filtering by region, product category, and performance tier.

Final Insights & Recommendations

- Highlights vendors causing large losses (e.g., -₹52,000 gross profit).
- Suggests procurement strategy changes and vendor renegotiations.

Vendor Performance Analytics: End-to-End Report

This document details an end-to-end data analytics project focused on evaluating and optimizing vendor performance using a multi-tool approach.

1. Project Overview & Objective

The primary goal of this project was to **analyze vendor performance to identify inefficiencies and optimize operations**. This analysis provides actionable recommendations via comprehensive dashboards and reports to improve vendor management.

2. Methodology and Data Workflow

The project followed a structured data analytics workflow:

Step	Tool/Technology	Description
------	-----------------	-------------

Data Source	SQL Database	Vendor transactions and raw performance metrics were sourced.
Data Cleaning & Feature Engineering	Python (Pandas)	Removed duplicates, handled missing values (forward fill), and standardized formats. Feature engineering involved calculating key metrics like Delivery_Duration (days between order and delivery) and Defect_Rate (Defective Items / Total Items).
Exploratory Data Analysis (EDA)	Python	Identified trends and outliers, and calculated initial Key Performance Indicators (KPIs) such as delivery time and defect rates.
Data Aggregation	SQL	Created summary tables for aggregated insights, such as vendor-wise performance statistics.
Visualization & Reporting	Power BI	Designed interactive dashboards, including KPI overviews, vendor comparisons, and actionable alerts.

3. Key Insights and Analysis

Insight Category	Description	Analytical Tool
Performance Ranking	Ranked Top Vendors based on favorable delivery time and quality scores.	Power BI, SQL
Underperformance	Identified	Python, Power BI

	Underperforming Vendors exhibiting high defect rates or significant delays.	
Trends	Observed seasonal variations in overall vendor performance.	Python EDA

Visualization Highlights (Python - Matplotlib/Seaborn):

- **Bar Plot:** Compared vendors by their **Average Delivery Duration**, highlighting consistently slow or fast performers.
- **Scatter Plot:** Visualized the relationship between **Defect Rate (%)** and **Delivery Duration (Days)** for all vendors. Vendors with high values on both axes (top-right cluster) were flagged for immediate attention.

4. Recommendations

Based on the analysis, the following recommendations were made to optimize vendor management:

- **Vendor Strategy:** Replace or renegotiate terms with identified underperforming vendors.
- **Incentivization:** Incentivize and reward top vendors to ensure sustained quality and performance.
- **Automation:** Automate data monitoring processes to provide real-time performance insights.

5. Tools Used Summary

- **SQL:** Data querying, aggregation, and initial data transformation.
- **Python:** Data cleaning, feature engineering, statistical analysis, and initial visualization.
- **Power BI:** Dynamic dashboard creation, interactive reporting, and final business presentation.

-----Appendix: Sample Python Code for Cleaning & VisualizationData Cleaning and Feature Engineering Script (Python)

```
import pandas as pd
```

```
# Load dataset
```

```

data = pd.read_csv("vendor_data.csv")

# Step 1: Handle Missing Values
data.fillna(method='ffill', inplace=True) # Forward fill

# Step 2: Create New Features
data['Delivery_Duration'] = (pd.to_datetime(data['Delivery_Date']) -
pd.to_datetime(data['Order_Date'])).dt.days
data['Defect_Rate'] = (data['Defective_Items'] / data['Total_Items']) * 100

# Step 3: Filter Relevant Data
filtered_data = data[data['Delivery_Duration'] > 0] # Remove invalid durations

# Step 4: Save Cleaned Data
filtered_data.to_csv("cleaned_vendor_data.csv", index=False)

```

Vendor Performance Visualization Script (Python)

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load cleaned dataset
data = pd.read_csv("cleaned_vendor_data.csv")

# Bar Plot for Average Delivery Duration per Vendor
avg_delivery = data.groupby('Vendor')['Delivery_Duration'].mean().sort_values()
plt.figure(figsize=(10, 6))
sns.barplot(x=avg_delivery.values, y=avg_delivery.index, palette="Blues_d")
plt.title("Average Delivery Duration by Vendor")
plt.xlabel("Average Delivery Duration (Days)")
plt.ylabel("Vendor")
plt.show()

# Scatter Plot for Defect Rate vs. Delivery Duration
plt.figure(figsize=(8, 6))
sns.scatterplot(data=data, x='Delivery_Duration', y='Defect_Rate', hue='Vendor',
palette="Set2", s=100)
plt.title("Defect Rate vs Delivery Duration")
plt.xlabel("Delivery Duration (Days)")
plt.ylabel("Defect Rate (%)")
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()

```


Project Summary: Vendor Performance Data Analytics

This project focuses on analyzing vendor performance using an end-to-end data analytics workflow involving **SQL, Python, and Power BI**. The objective was to identify inefficiencies, evaluate vendor reliability, and provide actionable recommendations to improve supply chain operations.

Objective

- Evaluate vendor performance using delivery efficiency, defect rates, and cost metrics.
 - Build interactive dashboards for data-driven decision-making.
 - Recommend actions to optimize vendor selection and performance.
-

Data Workflow Summary

1. Data Collection (SQL)

Vendor transactions, delivery records, and product-level metrics were extracted from SQL databases.

SQL helped clean, join, and aggregate raw tables to create vendor-wise summary data.

2. Data Cleaning & Feature Engineering (Python)

Using Pandas:

- Removed duplicates
- Handled missing values using forward fill
- Created new features:
 - **Delivery_Duration** = Delivery_Date – Order_Date
 - **Defect_Rate (%)** = Defective_Items / Total_Items × 100
- Filtered invalid or negative durations

- Exported cleaned dataset for analysis

3. Exploratory Data Analysis (Python)

EDA identified:

- Trends in delivery delays
- Outliers in defect rates
- Seasonal shifts in vendor performance
- KPIs such as average delivery time, defect rate, and cost ratios

Visualization included:

- **Bar Plot** → Average Delivery Duration per Vendor
 - **Scatter Plot** → Defect Rate vs Delivery Duration to detect underperformance
-



Power BI Dashboard Summary

The dashboard included:

- **KPI Cards** (delivery time, defect rate, efficiency)
- **Vendor Comparison Visuals**
- **Actionable Alerts** for poor-performing vendors
- Filters for region, category, and vendor tier

This allowed stakeholders to quickly identify risk areas and track vendor health.



Key Insights

- Some vendors consistently delivered fast with low defects (Top Vendors).

- A few vendors showed high defect rates and long delivery durations (Underperforming).
 - Seasonal patterns impacted vendor efficiency at certain periods.
 - Some vendors caused major financial losses (e.g., negative gross profit).
-



Recommendations

- **Renegotiate or replace** vendors with high delays or defect rates.
 - **Incentivize high-performing vendors** to maintain quality.
 - **Automate real-time monitoring** to detect issues early.
 - Improve procurement planning based on seasonal performance patterns.
-



Tools Used

- **SQL** → Data extraction & aggregation
 - **Python (Pandas, Matplotlib, Seaborn)** → Cleaning, EDA, visualizations
 - **Power BI** → Interactive dashboards and reporting
-



Final Summary

This project demonstrates a complete vendor analytics pipeline — from raw data extraction in SQL, cleaning and feature engineering in Python, to interactive dashboard creation in Power BI.

The insights help businesses optimize procurement strategy, reduce delays, lower defect rates, and enhance overall vendor reliability.

Thank You!!!